

Part 2: Elementary Data Structures Implementation and Discussion

1. Objective & Core Purpose

The primary goal of Step 2 is to evaluate the runtime efficiency, memory consumption, and structural tradeoffs of the foundational data structures implemented (**Custom Array**, **Matrix**, **Array Stack**, **Circular Array Queue**, **Singly Linked List**, and **Rooted Tree**).

By combining Theoretical Big-O Analysis with Empirical Benchmarking, Step 2 validates whether real-world execution times align with theoretical upper bounds across small, medium, and large input scale sizes (N).

1. Arrays and Matrices Implementation

We implement a fixed-capacity CustomArray (handling element shifting for insertion/deletion) and a row-major 2D Matrix.

Code:

```
1: from typing import Any, List

# 1. CUSTOM ARRAY (Fixed Capacity, Contiguous Memory)

class CustomArray: # Fixed-capacity array with explicit manual shifting for insertion and deletion.
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.size = 0
        self.data = [None] * capacity

    def access(self, index: int) -> Any: #Access: O(1) direct lookup by index.
        if not 0 <= index < self.size:
            raise IndexError("Array index out of bounds.")
        return self.data[index]

    def insert(self, index: int, value: Any) -> None: #Insertion: O(n) worst-case due to rightward element shifting.
        if self.size == self.capacity:
            raise MemoryError("Array capacity reached.")
        if not 0 <= index <= self.size:
            raise IndexError("Array index out of bounds.")

        # Shifting elements to the right to create space at index
        for i in range(self.size, index, -1):
            self.data[i] = self.data[i - 1]

        self.data[index] = value
        self.size += 1

    def delete(self, index: int) -> Any: #Deletion: O(n) worst-case due to leftward element shifting.
        if not 0 <= index < self.size:
            raise IndexError("Array index out of bounds.")

        removed_value = self.data[index]

        # Shifting elements to the left to close the gap
        for i in range(index, self.size - 1):
            self.data[i] = self.data[i + 1]

        self.data[self.size - 1] = None
        self.size -= 1
        return removed_value

# 2. MATRIX (2D Grid Structure)

class Matrix: #2D Row-Major Matrix supporting Insertion (Write), Deletion (Reset), and Access.
    def __init__(self, rows: int, cols: int, default_val: Any = 0):
        self.rows = rows
        self.cols = cols
        self.data = [[default_val for _ in range(cols)] for _ in range(rows)]

    def access(self, row: int, col: int) -> Any: #Access: O(1) lookup at coordinate (row, col).
        if not (0 <= row < self.rows and 0 <= col < self.cols):
            raise IndexError("Matrix coordinates out of bounds.")
        return self.data[row][col]

    def insert(self, row: int, col: int, value: Any) -> None: #Insertion/Update: O(1) write to coordinate (row, col)
        if not (0 <= row < self.rows and 0 <= col < self.cols):
            raise IndexError("Matrix coordinates out of bounds.")
        self.data[row][col] = value

    def delete(self, row: int, col: int, default_val: Any = 0) -> Any: #Deletion: O(1) reset of cell value at coordi
        if not (0 <= row < self.rows and 0 <= col < self.cols):
            raise IndexError("Matrix coordinates out of bounds.")
        removed_val = self.data[row][col]
        self.data[row][col] = default_val
        return removed_val
```

Validation

```
print(f"Deleted element at (row 0, col 1) (reset value back from {deleted_mat_val} to 0) ")
print("Matrix state after deletion:")
for r in range(mat.rows):
    print(f" Row {r}: {[mat.access(r, c) for c in range(mat.cols)]}")
```

1. CUSTOM ARRAY DEMONSTRATION

Array after insertions:
Contents: [10, 20, 30]

Accessing element at index 1: 20

Deleted element at index 1 (value: 20)

Array after deletion:
Contents: [10, 30]

2. MATRIX DEMONSTRATION

Matrix state after insertions:

Row 0: [0, 42, 0]

Row 1: [0, 0, 0]

Row 2: [0, 0, 99]

Accessing element at (Row 0, Col 1): 42

Deleted element at (Row 0, Col 1) (reset value back from 42 to 0)

Matrix state after deletion:

Row 0: [0, 0, 0]

Row 1: [0, 0, 0]

Row 2: [0, 0, 99]

Summary:

Structural Mechanics & Operations

- **Custom Array:** Represented over a raw fixed-capacity list buffer (self.data).
 - **Access $O(1)$:** Direct index lookup.
 - **Insertion $O(n)$:** Requires bounds checking and a backward loop shifting all elements from index i to $n - 1$ one position right to create a slot at index.
 - **Deletion $O(n)$:** Removes the value at index and shifts all trailing elements left by one position to maintain contiguous storage.
- **Matrix:** Implemented as a 2D grid storing row-major contiguous blocks. Access and update operations map directly to row and column coordinates in $O(1)$ time.

2. Stacks and Queues:

For the ArrayQueue, we implement a Circular Array using front and rear pointers so that enqueue and dequeue operations run in $O(1)$ time without requiring $O(n)$ element shifting.

Code:

```

from typing import Any, List

# 1. ARRAY-BASED STACK (LIFO: Last-In, First-Out)
class ArrayStack: #Stack implementation using a dynamic array base.
    def __init__(self):
        self.items: List[Any] = []

    def push(self, element: Any) -> None: #Insertion: Push element onto the top of the stack - O(1) amortized.
        self.items.append(element)

    def pop(self) -> Any: #Deletion: Remove and return the top element - O(1).
        if self.is_empty():
            raise IndexError("Stack Underflow: Cannot pop from an empty stack.")
        return self.items.pop()

    def peek(self) -> Any: #Access: View the top element without removing it - O(1).
        if self.is_empty():
            raise IndexError("Stack Empty: Cannot peek.")
        return self.items[-1]

    def is_empty(self) -> bool: #Check if the stack has no elements.
        return len(self.items) == 0

    def size(self) -> int: #Return total number of elements in the stack.
        return len(self.items)

# 2. CIRCULAR ARRAY-BASED QUEUE (FIFO: First-In, First-Out)
class CircularArrayQueue: #Queue implementation using a fixed-capacity circular array to achieve O(1) operations.
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.data = [None] * capacity
        self.front = 0
        self.rear = 0
        self.count = 0

    def enqueue(self, element: Any) -> None: #Insertion: Add element to the rear of the queue - O(1).
        if self.count == self.capacity:
            raise MemoryError("Queue Overflow: Capacity reached.")
        self.data[self.rear] = element
        self.rear = (self.rear + 1) % self.capacity
        self.count += 1

    def dequeue(self) -> Any: #Deletion: Remove and return the front element - O(1).
        if self.is_empty():
            raise IndexError("Queue Underflow: Cannot dequeue from an empty queue.")
        removed_val = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return removed_val

    def peek(self) -> Any: #Access: View the front element without removing it - O(1).
        if self.is_empty():
            raise IndexError("Queue Empty: Cannot peek.")
        return self.data[self.front]

    def is_empty(self) -> bool: #Check if the queue has no elements.
        return self.count == 0

    def size(self) -> int: #Return total number of elements in the queue.
        return self.count

```

Output:

```

1. ARRAY STACK DEMONSTRATION (LIFO)
Pushing elements: 10, 20, 30 onto stack...
Current Stack Size: 3
Accessing top element (peek): 30
Popped top element (deletion): 30
New top element after pop: 20
Stack size after pop: 2

2. CIRCULAR ARRAY QUEUE DEMONSTRATION (FIFO)
Enqueueing elements: 'Task A', 'Task B', 'Task C' into queue...
Current Queue Size: 3
Accessing front element (peek): 'Task A'
Dequeued front element (deletion): 'Task A'
New front element after dequeue: 'Task B'
Queue size after dequeue: 2

Enqueueing 'Task D' and 'Task E' to test circular buffer wrap...
Current Front element: 'Task B'
Total elements queued: 4/4

```

Linked list

Summary:

Structural Mechanics & Operations

- **Array Stack (LIFO):** Pushes and pops occur strictly at the end of the array, operating in $O(1)$ amortized time without element shifting.
- **Array Queue (FIFO - Circular):** Naive array dequeuing causes $O(n)$ shifts. To guarantee true $O(1)$ **enqueue** and **dequeue** operations, this implementation uses a fixed array with front and rear pointers updated modulo capacity:

$$\text{rear} = (\text{rear} + 1) \bmod \text{capacity}$$

3. Linked list

A SinglyLinkedList composed of explicit Node pointers with support for head/tail insertions, value deletion, and list traversal.

Code:

```
from typing import Any, List, Optional

# 1. NODE BUILDING BLOCK
class Node: #Individual node holding data and a pointer to the next node.
    def __init__(self, value: Any):
        self.value = value
        self.next: Optional['Node'] = None

# 2. SINGLY LINKED LIST DATA STRUCTURE
class SinglyLinkedList: #Singly Linked List with explicit pointer-manipulation methods.
    def __init__(self):
        self.head: Optional[Node] = None

    def insert_head(self, value: Any) -> None: #Insertion at Head: O(1) time complexity.
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node

    def insert_tail(self, value: Any) -> None: #Insertion at Tail: O(n) time complexity (requires traversing to the
        new_node = Node(value)
        if not self.head:
            self.head = new_node
            return

        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    def delete_value(self, value: Any) -> bool: #Deletion by Value: O(n) search time, O(1) pointer updates upon loc
        if not self.head:
            return False

        # Case 1: Target node is the head node
        if self.head.value == value:
            self.head = self.head.next
            return True

        # Case 2: Target node is downstream
        curr = self.head
        while curr.next and curr.next.value != value:
            curr = curr.next

        if curr.next: # Match found
            curr.next = curr.next.next
            return True

        return False # Target value not found in list

    def traverse(self) -> List[Any]: #Traversal: Visits every node from head to tail - O(n).
        elements = []
        curr = self.head
        while curr:
            elements.append(curr.value)
            curr = curr.next
        return elements

    def search(self, value: Any) -> bool: #Access/Search: Determines if a value exists - O(n).
        curr = self.head
        while curr:
            if curr.value == value:
                return True
            curr = curr.next
        return False
```

Output:

```

SINGLY LINKED LIST DEMONSTRATION
1. INSERTION OPERATIONS:
-> Inserting 20 at head
-> Inserting 10 at head
-> Inserting 30 at tail
-> Inserting 40 at tail

2. TRAVERSAL OPERATION:
Current List Sequence: [10, 20, 30, 40]

3. SEARCH / ACCESS OPERATIONS:
Is 30 in the list? True
Is 99 in the list? False

4. DELETION OPERATIONS:
-> Deleting middle element (20)...
Updated List: [10, 30, 40]
-> Deleting head element (10)...
Updated List: [30, 40]
-> Attempting to delete non-existent element (99)...
Deletion status: False
Final List State: [30, 40]

Rooted Trees

```

Summary: Structural Mechanics & Operations

- **Node Links:** Each node stores a value payload and a next pointer pointing to the downstream node or None.
- **Head Insertion $O(1)$:** Creates a node, points its next to current head, and re-assigns head.
- **Tail Insertion & Value Deletion $O(n)$:** Requires traversing the pointer chain using a curr reference until reaching the target node or list tail, then splicing next pointers.

4. Rooted Trees

A general N-ary Rooted Tree implementation using nodes linked to lists of child node pointers, along with a Depth-First Search (DFS) traversal.

```

1: from typing import Any, List, Optional
from collections import deque

# 1. TREE NODE (Linked Children Structure)
class TreeNode: #Node structure for general rooted trees using linked child references.
    def __init__(self, value: Any):
        self.value = value
        self.children: List['TreeNode'] = []

    def add_child(self, child_node: 'TreeNode') -> None: #Insertion: Adds a child reference link to this node - O(1)
        self.children.append(child_node)

    def remove_child(self, target_value: Any) -> bool: #Deletion: Removes a child node and its entire subtree - O(c)
        for idx, child in enumerate(self.children):
            if child.value == target_value:
                del self.children[idx]
                return True
        return False

# 2. ROOTED TREE WRAPPER
class RootedTree: #General Rooted Tree operations wrapper.
    def __init__(self, root_val: Any):
        self.root = TreeNode(root_val)

    def insert(self, parent_val: Any, new_val: Any) -> bool: #Insertion: Locates target parent node and appends a new child
        parent_node = self.find(self.root, parent_val)
        if parent_node:
            parent_node.add_child(TreeNode(new_val))
            return True
        return False

    def delete(self, value: Any) -> bool: #Deletion: Prunes target subtree starting at matching value.
        if self.root.value == value:
            raise ValueError("Cannot delete the root node via prune operation.")
        return self._delete_helper(self.root, value)

    def _delete_helper(self, current: TreeNode, value: Any) -> bool:
        if current.remove_child(value):
            return True
        for child in current.children:
            if self._delete_helper(child, value):
                return True
        return False

    def find(self, current: Optional[TreeNode], value: Any) -> Optional[TreeNode]: #Access / Search: Recursively locates node
        if not current:
            return None
        if current.value == value:
            return current
        for child in current.children:
            res = self.find(child, value)
            if res:
                return res
        return None

    def traverse_dfs(self, current: Optional[TreeNode] = None) -> List[Any]: #Traversal (DFS): Pre-order recursive depth-first search
        if current is None:
            current = self.root
        result = [current.value]
        for child in current.children:
            result.extend(self.traverse_dfs(child))
        return result

    def traverse_bfs(self) -> List[Any]: #Traversal (BFS): Level-order queue-based breadth-first exploration - O(n).
        if not self.root:
            return result
        queue = deque([self.root])
        while queue:
            curr = queue.popleft()
            result.append(curr.value)
            for child in curr.children:
                queue.append(child)
        return result

```

Output:

```
ROOTED TREE DEMONSTRATION
1. CREATING TREE & INSERTING NODES...
   Inserted: 'FolderA' & 'FolderB' under 'Root'
   Inserted: 'FileA1' & 'FileA2' under 'FolderA'

2. TREE TRAVERSALS:
   Depth-First Search (DFS - Pre-order): ['Root', 'FolderA', 'FileA1', 'FileA2', 'FolderB']
   Breadth-First Search (BFS - Level-order): ['Root', 'FolderA', 'FolderB', 'FileA1', 'FileA2']

3. SEARCH / ACCESS OPERATIONS:
   Search for 'FileA1': Found node with value 'FileA1'
   Search for non-existent 'FileZ': Found -> None

4. DELETION (SUBTREE PRUNING):
   -> Deleting node 'FileA2'...
       DFS Traversal after deleting 'FileA2': ['Root', 'FolderA', 'FileA1', 'FolderB']
   -> Deleting entire subtree 'FolderA'...
       Final DFS Traversal (FolderA and its contents removed): ['Root', 'FolderB']
```

Summary:

Structural Mechanics & Operations

- **Node Hierarchy:** Stored as `TreeNode` objects carrying a payload and a children list holding reference pointers to child nodes.
- **Traversal:** Implements recursive Depth-First Search (DFS) to explore parent nodes followed by each child branch sequentially.

2. Performance Analysis

Time Complexity Summary

The table below summarizes the asymptotic time complexity for the fundamental operations across each implemented data structure.

Data Structure	Access	Search	Insertion	Deletion
Array / Matrix	$O(1)$	$O(n)$	$O(n)$ (requires element shifting)	$O(n)$ (requires element shifting)
Stack (Array-based)	$O(n)$	$O(n)$	$O(1)$ amortized (push at top)	$O(1)$ (pop from top)
Queue (Circular Array)	$O(n)$	$O(n)$	$O(1)$ (enqueue at rear)	$O(1)$ (dequeue from front)
Singly Linked List	$O(n)$	$O(n)$	$O(1)$ at Head / $O(n)$ at Tail	$O(1)$ at Head / $O(n)$ by Value
Rooted Tree (N-ary)	$O(n)$	$O(n)$	$O(1)$ (append child to node)	$O(n)$ (node/subtree removal)

Trade-offs: Array vs. Linked List Implementations

1. Stacks (LIFO)

- **Array-Based Stack:**

- **Pros:** Exceptional CPU cache locality because elements sit in contiguous memory. Minimal memory overhead per element (no pointer storage).
- **Cons:** Dynamic arrays incur an occasional $O(n)$ latency cost during array resizing/reallocation when capacity limits are hit.
- **Linked-List-Based Stack:**
 - **Pros:** Strict, deterministic $O(1)$ worst-case time for every push and pop operation—no periodic reallocation pauses.
 - **Cons:** Higher memory overhead per item due to storing next node pointers alongside data values. Poor CPU cache utilization due to non-contiguous heap memory allocations.

2. Queues (FIFO)

- **Array-Based Queue:**
 - **Naive Array Approach:** Enqueueing at the end is $O(1)$, but dequeuing from index 0 requires shifting all trailing elements left, leading to an inefficient $O(n)$ time complexity.
 - **Circular Array Approach:** By using front and rear pointers with modulo arithmetic $((\text{index} + 1) \% \text{capacity})$, both enqueue and dequeue achieve true $O(1)$ time in fixed memory without element shifting.
- **Linked-List-Based Queue:**
 - **Pros:** Adding a tail reference to a Singly Linked List allows $O(1)$ enqueue (at tail) and $O(1)$ dequeue (at head) without needing a predetermined max capacity.
 - **Cons:** Higher memory consumption per queued element and overhead from continuous node object instantiation/garbage collection.

Scenario-Based Efficiency Comparison

To select the optimal data structure, consider these common practical constraints:

Scenario A: High-Frequency Random Access & Mathematical Computations

- **Best Choice: Arrays / Matrices**
- **Why:** In image processing or matrix multiplication algorithms, accessing elements by spatial index (r,c) must be instantaneous $O(1)$. Arrays leverage hardware memory prefetching and SIMD instructions, making them orders of magnitude faster than linked lists for numeric sweeps.

Scenario B: Arbitrary Insertions & Deletions in Large Datasets

- **Best Choice: Linked Lists**
- **Why:** If we already hold an iterator/pointer to a specific target location inside a list, inserting or removing a node requires updating only two pointer references $O(1)$. In an array, inserting or deleting in the middle requires physically shifting $O(n)$ elements in memory.

Scenario C: Fixed-Capacity Buffering (Streaming Systems)

- **Best Choice: Circular Array Queue**
- **Why:** Telecommunications and network packet processing rely on predictable, bounded memory. A circular array queue provides deterministic $O(1)$ arrival/dispatch performance without risk of unconstrained dynamic memory expansion.

3. Discussion & Practical Applications

Real-World Scenarios for Elementary Data Structures

- Each elementary data structure plays a vital role in computer systems and software applications.

1. Arrays and Matrices

- **Real-World Applications:** Image processing (2D pixel grids), numerical computing libraries (e.g., NumPy), game development (coordinate grids, spatial matrices), and audio signal processing.
- **Why Preferred:** Continuous memory allocation enables $O(1)$ **random access** by index. CPU memory controllers prefetch adjacent array elements into the cache, providing maximum memory bandwidth and enabling hardware-level SIMD (Single Instruction, Multiple Data) vectorization.

2. Stacks (LIFO Last-In, First-Out)

- **Real-World Applications:** Function call stack execution in compilers/runtimes, expression parsing and syntax evaluation (e.g., Dijkstra's Shunting-yard algorithm), Undo/Redo operations in text editors, and Depth-First Search (DFS) graph traversals.
- **Why Preferred:** Enforces strict structural discipline. By restricting all mutations exclusively to the top of the structure, stacks eliminate unauthorized middle-element access and simplify state backtracking.

3. Queues (FIFO — First-In, First-Out)

- **Real-World Applications:** Operating system task scheduling (e.g., CPU round-robin thread queues), network packet buffering (routers/switches), printer job spooling, web server asynchronous HTTP request handling, and Breadth-First Search (BFS) graph traversals.
- **Why Preferred:** Preserves exact temporal order, guaranteeing fair processing without starvation for incoming requests.

4. Linked Lists

- **Real-World Applications:** Low-level OS kernel memory allocation (free memory block tracking via explicit free lists), implementation of LRU (Least Recently Used) caches

(doubly linked lists combined with hash maps), and media player playlists (previous/next song navigation).

- **Why Preferred:** Essential when element insertions and deletions must occur at arbitrary positions in $O(1)$ **time** once a node pointer is available, without incurring array relocation or element shifting penalties.

5. Rooted Trees

- **Real-World Applications:** Operating system file systems (directory/folder trees), HTML/XML Document Object Model (DOM) rendering engines in web browsers, Abstract Syntax Trees (ASTs) in compilation, and JSON/YAML document parsing.
- **Why Preferred:** Provides a natural mathematical model for representing non-linear, hierarchical relationships between parent and child elements.

Key Trade-Off Matrix

When choosing between these data structures, balance these three core software engineering criteria:

Requirement / Factor	Dominant Data Structure	Trade-off Rationale
Raw Execution Speed & Cache Performance	Contiguous Arrays	Outperforms pointer-based structures due to spatial locality and CPU L1/L2 cache hits, despite $O(n)$ insertion shifting costs.
Predictable Memory Footprint	Circular Array Stacks/Queues	Eliminates pointer overhead (8 bytes per node on 64-bit systems) and prevents heap fragmentation.
Dynamic Sizing & Low Insertion Latency	Linked Lists	Eliminates large contiguous array allocation failures and occasional $O(n)$ re-allocation pauses, though sacrificing random access.
Hierarchical Modeling	Rooted Trees	Cannot be efficiently flattened into linear arrays without losing explicit structural parent-child relationships.

References:

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022).** Introduction to Algorithms (4th ed.). MIT Press.
- **Weiss, M. A. (2014).** Data Structures and Algorithm Analysis in Java (4th ed.) / C++ (4th ed.). Pearson.
- **Patterson, D. A., & Hennessy, J. L. (2020).** Computer Organization and Design RISC-V Edition: The Hardware Software Interface (2nd ed.). Morgan Kaufmann.